

Exam: Time Series Visualization with Bokeh

This exam tests your ability to visualize time series data using the Bokeh library. You will be working with the "Daily Minimum Temperatures in Melbourne" dataset. For each question, provide the Python code using Bokeh to generate the requested visualization.

Dataset: "daily-minimum-temperatures-in-melbourne.csv"

```
import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("daily-minimum-temperatures-in-melbourne.csv")

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])
```

```
In [7]: import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

print("Données chargées et outils Bokeh importés avec succès !")
df.head()
```



Loading BokehJS ...

Données chargées et outils Bokeh importés avec succès !

Out[7]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Question 1: Basic Time Series Line Plot

1. Create a basic line plot showing the daily minimum temperature over time.

- Use the 'Date' column on the x-axis and the 'Temperature' column on the y-axis.
- Set the plot title to "Daily Minimum Temperatures".
- Label the x-axis as "Date" and the y-axis as "Temperature (°C)".
- Add tooltips to display the date and temperature when hovering over the line.
- Enable pan, wheel zoom, and reset tools.

In [8]:

```
# Answer 1: Basic Time Series Line Plot
source1 = ColumnDataSource(df)

p1 = figure(title="Daily Minimum Temperatures", x_axis_label="Date", y_axis_label="Temperature (°C)",
            x_axis_type="datetime", width=800, height=400, tools="pan,wheel_zoom,reset")

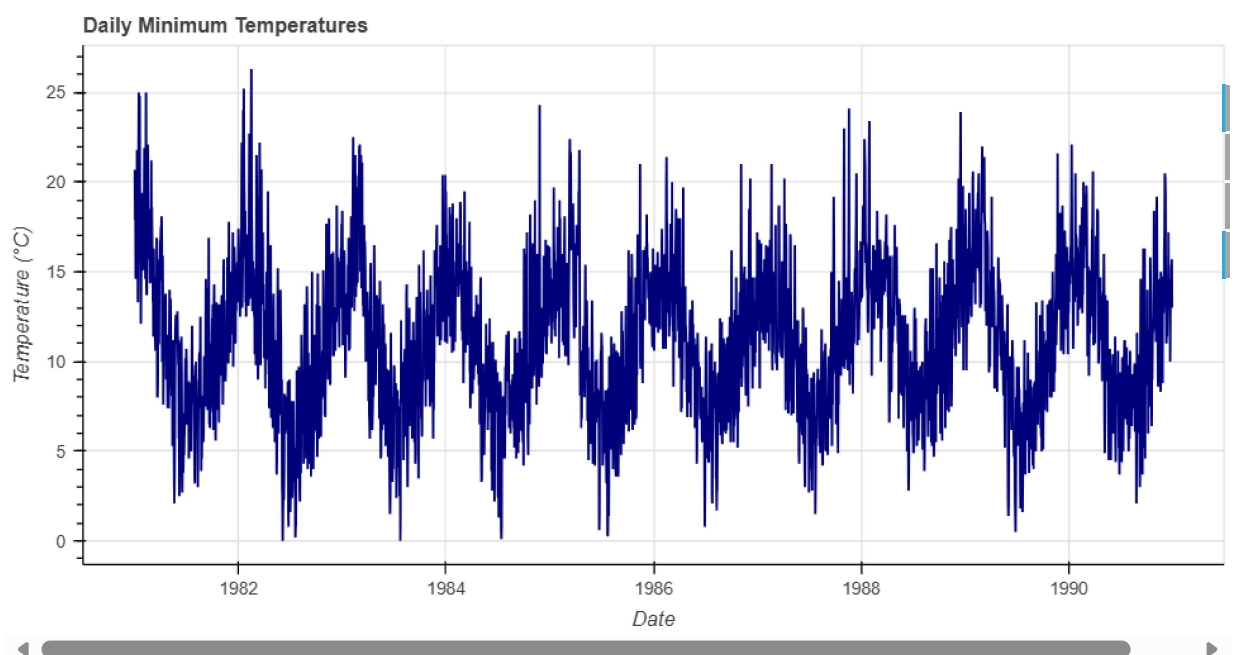
p1.line(x='Date', y='Temperature', source=source1, color="navy", line_width=1.5)

# Ajout du Tooltip
hover1 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Temperature", "@Temperature{0.0} °C")],
    formatters={'@Date': 'datetime'}
)
p1.add_tools(hover1)

from bokeh.io import output_file

# On demande à Bokeh de créer un vrai fichier web
output_file("question1.html")

# Et on l'affiche ! (Ça va ouvrir un nouvel onglet dans ton navigateur)
show(p1)
```



Question 2: Rolling Average 2. Calculate the 30-day rolling average of the daily minimum temperature and plot it alongside the original temperature data.

- * Create a new column 'Rolling_Avg' in the DataFrame containing the 30-day rolling average.
- * Plot both the original 'Temperature' and the 'Rolling_Avg' on the same plot.
- * Use different colors and line styles to distinguish between the two.
- * Add a legend to the plot to label the lines.
- * Add tooltips to display the date, original temperature, and rolling average.

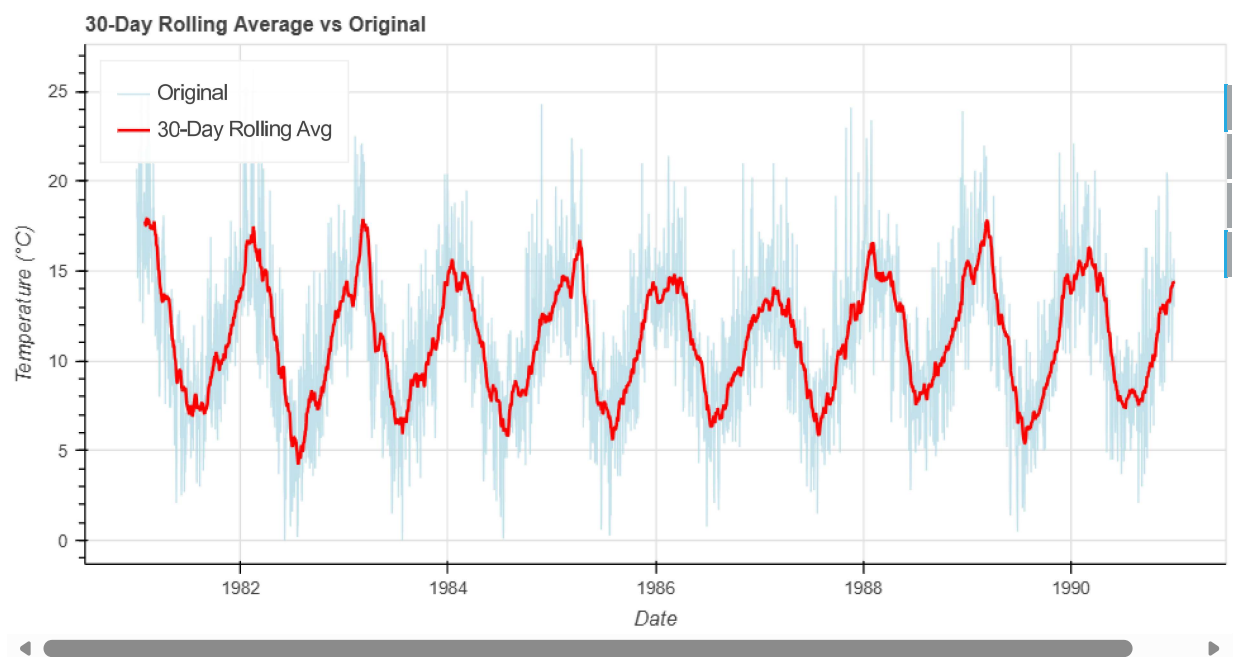
```
In [9]: # Answer 2: Rolling Average
# 1. Calcul de la moyenne mobile
df['Rolling_Avg'] = df['Temperature'].rolling(window=30).mean()
source2 = ColumnDataSource(df)

p2 = figure(title="30-Day Rolling Average vs Original", x_axis_label="Date", y_axis_label="Temperature (°C",
            x_axis_type="datetime", width=800, height=400, tools="pan,wheel_zoom,reset")

# Ligne originale (plus transparente) et Ligne de moyenne (rouge et épaisse)
p2.line('Date', 'Temperature', source=source2, color="lightblue", alpha=0.7, legend_label="Original")
p2.line('Date', 'Rolling_Avg', source=source2, color="red", line_width=2, legend_label="30-Day Rolling Avg")

hover2 = HoverTool(
    tooltips=[("Date", "@Date{%F}"), ("Original Temp", "@Temperature{0.0} °C"), ("Rolling Avg", "@Rolling_
    formatters={'@Date': 'datetime'})
)
p2.add_tools(hover2)
p2.legend.location = "top_left"

from bokeh.io import output_file
output_file("question2_rolling_avg.html")
show(p2)
```



Question 3: Monthly Box Plots 3. Create box plots to visualize the distribution of temperatures for each month.

- * Extract the month from the 'Date' column and create a new 'Month' column.
- * Group the data by 'Month' and prepare it for plotting.
- * Use Bokeh's box plot elements to visualize the distribution.
- * Label the x-axis with month names and the y-axis with "Temperature (°C)".
- * Add tooltips to display the month and relevant statistical values (min, max, media

```
In [10]: # Answer 3: Monthly Box Plots
# 1. Extraction du mois
df['Month'] = df['Date'].dt.month_name()
months = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October"]

# 2. Calcul des statistiques pour chaque mois
groups = df.groupby('Month')
q1 = groups['Temperature'].quantile(0.25)
q2 = groups['Temperature'].quantile(0.50) # Médiane
q3 = groups['Temperature'].quantile(0.75)
iqr = q3 - q1
```

```

upper = q3 + 1.5 * iqr
lower = q1 - 1.5 * iqr

# Création d'un dataset pour Bokeh
stats = pd.DataFrame(dict(q1=q1, q2=q2, q3=q3, upper=upper, lower=lower)).reindex(months)

# ---> LA LIGNE MAGIQUE POUR ÉVITER LE CONFLIT BOKEH <---
stats.index.name = None

stats['Month'] = stats.index
source3 = ColumnDataSource(stats)

p3 = figure(x_range=months, title="Monthly Temperature Distribution", x_axis_label="Month", y_axis_label="

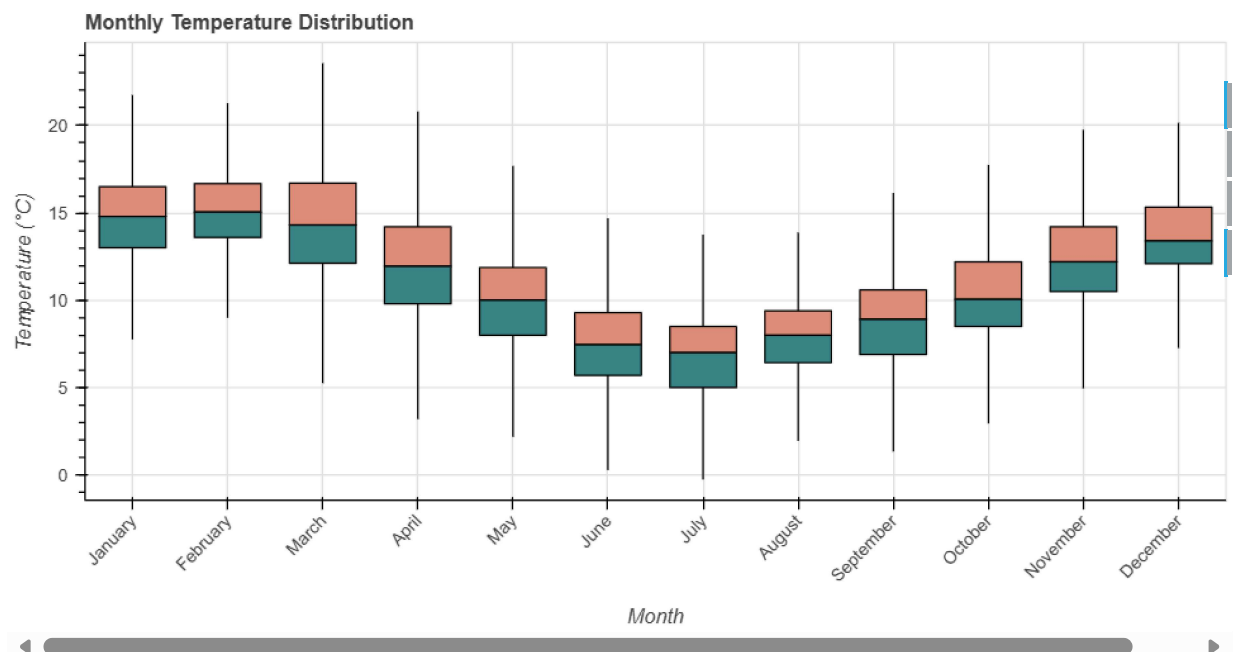
# Dessin des boîtes à moustaches (Tiges puis boîtes)
p3.segment('Month', 'upper', 'Month', 'q3', source=source3, line_color="black")
p3.segment('Month', 'lower', 'Month', 'q1', source=source3, line_color="black")
p3.vbar('Month', 0.7, 'q2', 'q3', source=source3, fill_color="#E08E79", line_color="black")
p3.vbar('Month', 0.7, 'q1', 'q2', source=source3, fill_color="#3B8686", line_color="black")

hover3 = HoverTool(tooltips=[("Month", "@Month"), ("Min", "@lower{0.0}"), ("Max", "@upper{0.0}"), ("Median

p3.add_tools(hover3)
p3.xaxis.major_label_orientation = 0.8

from bokeh.io import output_file
output_file("question3_boxplots_mois.html")
show(p3)

```



```

In [11]: from bokeh.plotting import output_notebook, show
import pandas as pd

output_notebook() # Enable Bokeh output in Jupyter Notebook

# Load the Dataset
df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Now you can proceed with your Bokeh plotting code!
print(df.head()) # Just to see if the dataframe loaded correctly

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

df

```



Loading BokehJS ...

	Date	DailyTemperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

Out[11]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8
...	...	...
3645	1990-12-27	14.0
3646	1990-12-28	13.6
3647	1990-12-29	13.5
3648	1990-12-30	15.7
3649	1990-12-31	13.0

3650 rows × 2 columns

4. Create box plots to visualize the distribution of temperatures for each year, and use color mapping to highlight temperature variations.

- Extract the year from the 'Date' column and create a new 'Year' column.
- Group the data by 'Year' and prepare it for plotting.
- Use Bokeh's box plot elements to visualize the distribution for each year.
- Label the x-axis with the 'Year' and the y-axis with "Temperature (°C)".
- Use `factor_cmap` to color the boxes based on the median temperature of each year.
- Add tooltips to display the year and relevant statistical values (min, max, median, etc.).
- Enable pan, wheel zoom, and reset tools.

```
In [12]: # Answer 4: Yearly Box Plots with Color Mapping
from bokeh.palettes import Spectral11

df['Year'] = df['Date'].dt.year.astype(str)
years = sorted(df['Year'].unique())

groups_yr = df.groupby('Year')
q1_y = groups_yr['Temperature'].quantile(0.25)
q2_y = groups_yr['Temperature'].quantile(0.50)
q3_y = groups_yr['Temperature'].quantile(0.75)
iqr_y = q3_y - q1_y
upper_y = q3_y + 1.5 * iqr_y
lower_y = q1_y - 1.5 * iqr_y

stats_yr = pd.DataFrame(dict(q1=q1_y, q2=q2_y, q3=q3_y, upper=upper_y, lower=lower_y)).reindex(years)

# ---> LA LIGNE MAGIQUE POUR ÉVITER L'ERREUR <---
stats_yr.index.name = None

stats_yr['Year'] = stats_yr.index
source4 = ColumnDataSource(stats_yr)

# Cartographie des couleurs (factor_cmap)
mapper = factor_cmap(field_name='Year', palette=Spectral11, factors=years)

p4 = figure(x_range=years, title="Yearly Temperature Distribution", x_axis_label="Year", y_axis_label="Tem
p4.segment('Year', 'upper', 'Year', 'q3', source=source4, line_color="black")
```

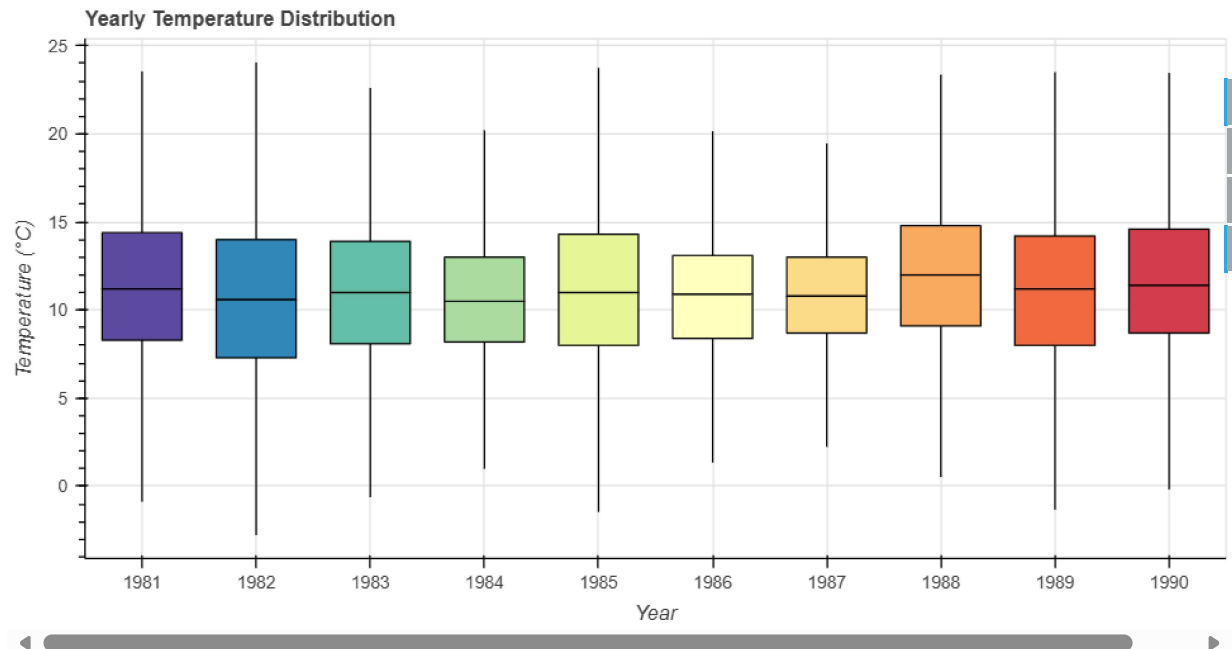
```

p4.segment('Year', 'lower', 'Year', 'q1', source=source4, line_color="black")
p4.vbar('Year', 0.7, 'q2', 'q3', source=source4, fill_color=mapper, line_color="black")
p4.vbar('Year', 0.7, 'q1', 'q2', source=source4, fill_color=mapper, line_color="black")

hover4 = HoverTool(tooltips=[("Year", "@Year"), ("Min", "@lower{0.0}"), ("Max", "@upper{0.0}"), ("Median",
p4.add_tools(hover4)

from bokeh.io import output_file
output_file("question4_boxplots_annee.html")
show(p4)

```



Question 5: Interactive Time Range Selection

5. Create an interactive line plot where the user can select a specific time range to view using a date range slider.

- Create a basic line plot of 'Temperature' over 'Date'.
- Implement a date range slider using Bokeh widgets to allow users to select a start and end date.
- Update the plot dynamically based on the selected date range.
- Add tooltips to display the date and temperature.
- Enable pan, wheel zoom, and reset tools.

```

In [13]: # Answer 5: Interactive Time Range Selection
from bokeh.models import DateRangeSlider
from bokeh.layouts import column

source5 = ColumnDataSource(df)

p5 = figure(title="Interactive Time Range (Use the slider below)", x_axis_label="Date", y_axis_label="Temp",
            x_axis_type="datetime", width=800, height=350, tools="pan,wheel_zoom,reset")

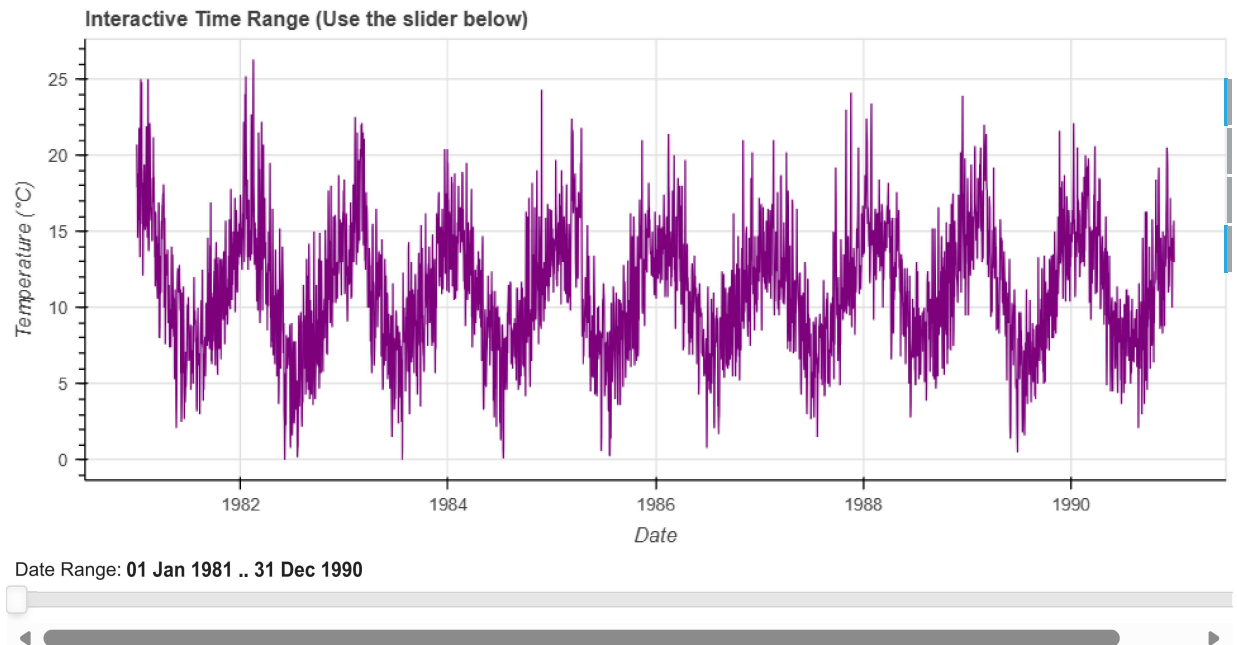
p5.line('Date', 'Temperature', source=source5, color="purple")
p5.add_tools(HoverTool(tooltips=[("Date", "@Date{%F}"), ("Temp", "@Temperature{0.0}"),], formatters={'@Date

# Création du curseur interactif Lié à L'axe X
date_slider = DateRangeSlider(title="Date Range", start=df['Date'].min(), end=df['Date'].max(),
                             value=(df['Date'].min(), df['Date'].max()), step=1, width=800)
date_slider.js_link('value', p5.x_range, 'start', attr_selector=0)
date_slider.js_link('value', p5.x_range, 'end', attr_selector=1)

from bokeh.io import output_file
output_file("question5_slider_interactif.html")

# On affiche la colonne contenant le graphique et le curseur
show(column(p5, date_slider))

```



Question 6: Time Series Decomposition Visualization

6. Perform a simple time series decomposition to visualize the trend and seasonality components of the temperature data.

- Resample the data to monthly frequency and calculate the monthly average temperature.
- Use a simple moving average to estimate the trend component.
- Calculate the seasonal component by subtracting the trend from the original monthly data.
- Create three separate Bokeh plots: one for the original monthly data, one for the trend, and one for the seasonal component.
- Ensure the plots are aligned and share the same x-axis (Date).
- Add tooltips to each plot to display the date and corresponding value.
- Enable pan, wheel zoom, and reset tools for each plot.

```
In [14]: # Answer 6: Time Series Decomposition Visualization
# 1. Rééchantillonnage et mathématiques
monthly_df = df.set_index('Date').resample('ME')['Temperature'].mean().reset_index()
monthly_df['Trend'] = monthly_df['Temperature'].rolling(window=12, center=True).mean()
monthly_df['Seasonality'] = monthly_df['Temperature'] - monthly_df['Trend']
source6 = ColumnDataSource(monthly_df)

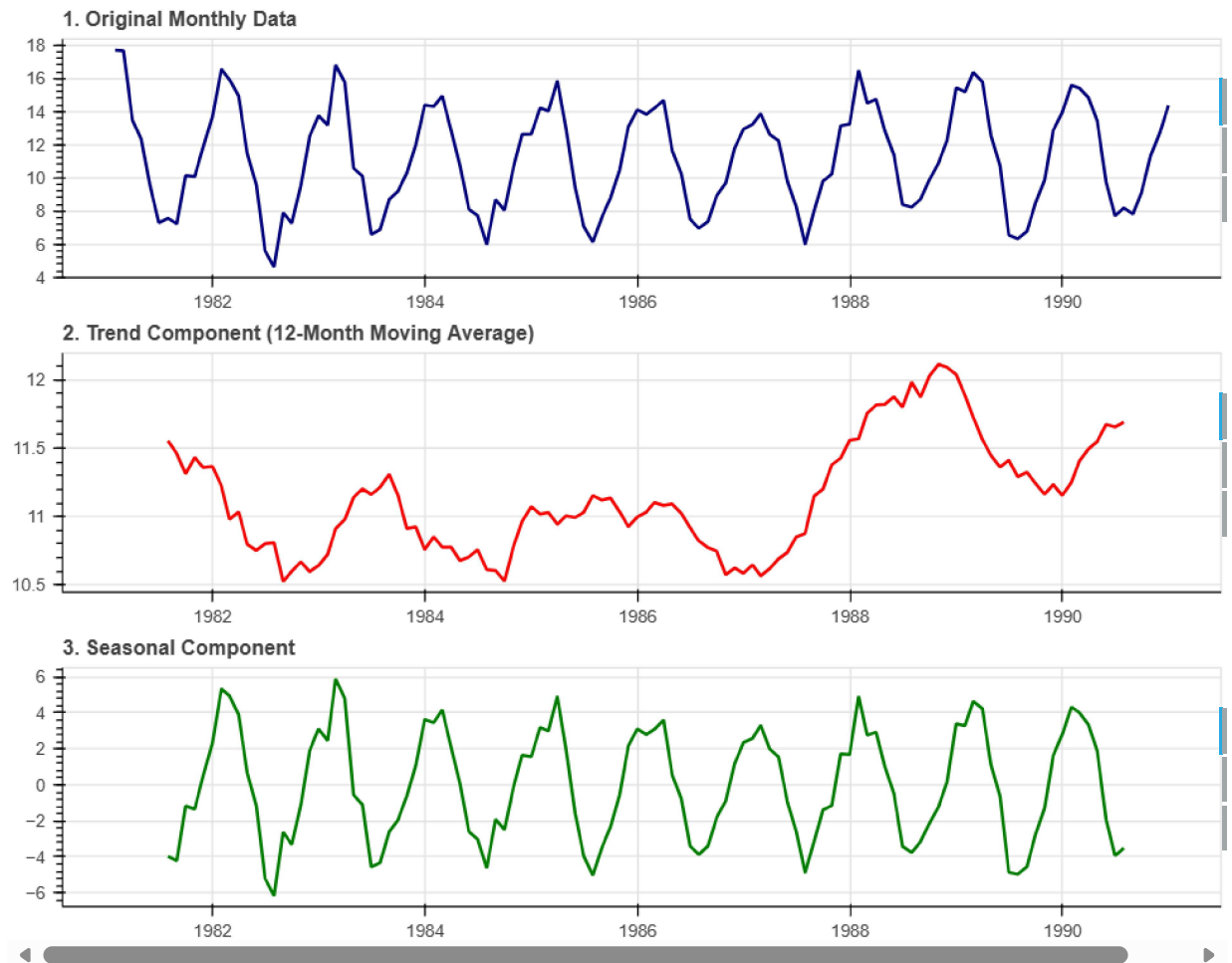
# 2. Graphique 1 : Original
p_orig = figure(title="1. Original Monthly Data", x_axis_type="datetime", width=800, height=200, tools="pan")
p_orig.line('Date', 'Temperature', source=source6, color="navy", line_width=2)
p_orig.add_tools(HoverTool(tooltips=[("Date", "@Date{%F}"), ("Temp", "@Temperature{0.0}")] , formatters={"@Date": "@Date{%F}"})

# 3. Graphique 2 : Tendence (Partage L'axe X avec Le graphique 1)
p_trend = figure(title="2. Trend Component (12-Month Moving Average)", x_axis_type="datetime", width=800, height=200, tools="pan")
p_trend.line('Date', 'Trend', source=source6, color="red", line_width=2)
p_trend.add_tools(HoverTool(tooltips=[("Date", "@Date{%F}"), ("Trend", "@Trend{0.0}")] , formatters={"@Date": "@Date{%F}"})

# 4. Graphique 3 : Saisonnalité (Partage L'axe X avec Le graphique 1)
p_seas = figure(title="3. Seasonal Component", x_axis_type="datetime", width=800, height=200, x_range=p_orig.x_range, tools="pan")
p_seas.line('Date', 'Seasonality', source=source6, color="green", line_width=2)
p_seas.add_tools(HoverTool(tooltips=[("Date", "@Date{%F}"), ("Seasonality", "@Seasonality{0.0}")] , formatters={"@Date": "@Date{%F}"})

from bokeh.io import output_file
output_file("question6_decomposition_meteo.html")

# On affiche Les 3 graphiques empilés
show(column(p_orig, p_trend, p_seas))
```



```
In [16]: import pandas as pd
from bokeh.plotting import figure, show
from bokeh.io import output_notebook
from bokeh.models import (
    ColumnDataSource,
    HoverTool,
    DatetimeTickFormatter,
    NumeralTickFormatter,
)
from bokeh.layouts import row, column
from bokeh.transform import factor_cmap

output_notebook() # Enable Bokeh output in Jupyter Notebook

df = pd.read_csv("../datasets/daily-minimum-temperatures-in-melbourne.csv")

# Rename columns for clarity
df.columns = ['Date', 'Temperature']

# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Remove '?' from the 'Temperature' column and convert to numeric
df['Temperature'] = df['Temperature'].astype(str).str.replace('?', '', regex=False)
df['Temperature'] = pd.to_numeric(df['Temperature'])

df
```



BokehJS 3.8.2 successfully loaded.

Out[16]:

	Date	Temperature
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8
...	...	...
3645	1990-12-27	14.0
3646	1990-12-28	13.6
3647	1990-12-29	13.5
3648	1990-12-30	15.7
3649	1990-12-31	13.0

3650 rows × 2 columns